File - /Users/mazenfraihat/IdeaProjects/HW10/src/main/java/part2/User.java

```java
package part2;

import java.util.Iterator;
import java.util.List;

public class User implements IterableByUser {
    private String name;
    private ChatServer server;
    private ChatHistory history;

    public User(String name, ChatServer server) {
        this.name = name;
        this.server = server;
        this.history = new ChatHistory();
    }

    public String getName() {
        return name;
    }

    public ChatHistory getChatHistory() {
        return history;
    }

    public void sendMessage(String content, List<User> recipients) {
        Message message = new Message(this, recipients, content);
        history.addMessage(message);
        history.saveMemento(new MessageMemento(content, message.
getTimestamp()));
        server.sendMessage(message);
    }

    public void receiveMessage(Message message) {
        history.addMessage(message);
    }

    public void undoLastMessage() {
        MessageMemento memento = history.getLastMessageSent();
        if (memento == null) {
            System.out.println(name + " has nothing to undo.");
            return;
        }
        history.removeMessage(memento);
        server.undoMessage(this, memento);
        System.out.println(name + " undid: " + memento.getContent());
    }

    public void blockUser(User other) {
        server.blockUser(this, other);
        System.out.println(name + " blocked " + other.getName());
    }

    public void viewChatHistory() {
```

```
53          System.out.println("--- " + name + "'s chat history ---");
54          for (Message m : history.getMessages()) {
55              System.out.println(m);
56          }
57      }
58
59      public Iterator<Message> iterator(User userToSearchWith) {
60          return history.iterator(userToSearchWith);
61      }
62  }
```

```java
package part2;

import java.util.List;
import java.util.Iterator;

public class Driver {
    public static void main(String[] args) {

        System.out.println("=== 1. Setup: server + 3 users ===");
        ChatServer server = new ChatServer();
        User mazen = new User("Mazen", server);
        User ahmed = new User("Ahmed", server);
        User sara = new User("Sara", server);
        server.registerUser(mazen);
        server.registerUser(ahmed);
        server.registerUser(sara);
        System.out.println("Mazen, Ahmed, and Sara registered.");

        System.out.println("\n=== 2. Send to one user ===");
        mazen.sendMessage("salam", List.of(ahmed));

        System.out.println("\n=== 3. Send to many users ===");
        ahmed.sendMessage("meeting at 5", List.of(mazen, sara));

        System.out.println("\n=== 4. View chat history ===");
        mazen.viewChatHistory();
        ahmed.viewChatHistory();
        sara.viewChatHistory();

        System.out.println("\n=== 5. Undo last message (Memento) ===");
        mazen.sendMessage("oops wrong msg", List.of(ahmed, sara));
        System.out.println("Before undo:");
        ahmed.viewChatHistory();
        mazen.undoLastMessage();
        System.out.println("After undo:");
        mazen.viewChatHistory();
        ahmed.viewChatHistory();
        sara.viewChatHistory();

        System.out.println("\n=== 6. Block a user (Mediator) ===");
        sara.blockUser(mazen);
        mazen.sendMessage("hey", List.of(ahmed, sara));
        ahmed.viewChatHistory();
        sara.viewChatHistory();

        System.out.println("\n=== 7. Undo with nothing sent ===");
        sara.undoLastMessage();

        System.out.println("\n=== 8. Unregister a user ===");
        server.unregisterUser(ahmed);
        mazen.sendMessage("are you there?", List.of(ahmed));

```

```
53
54          System.out.println("\n=== 9. Iterate Mazen's history by user
    (Iterator) ===");
55          System.out.println("Messages with Ahmed:");
56          Iterator<Message> byAhmed = mazen.iterator(ahmed);
57          while (byAhmed.hasNext()) {
58              System.out.println(byAhmed.next());
59          }
60
61          System.out.println("Messages with Sara:");
62          Iterator<Message> bySara = mazen.iterator(sara);
63          while (bySara.hasNext()) {
64              System.out.println(bySara.next());
65          }
66      }
67  }
```

```java
package part2;

import java.time.LocalDateTime;
import java.util.List;

public class Message {
    private User sender;
    private List<User> recipients;
    private LocalDateTime timestamp;
    private String content;

    public Message(User sender, List<User> recipients, String content) {
        this.sender = sender;
        this.recipients = recipients;
        this.content = content;
        this.timestamp = LocalDateTime.now();
    }

    public User getSender() {
        return sender;
    }

    public List<User> getRecipients() {
        return recipients;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public String getContent() {
        return content;
    }

    public String toString() {
        String names = "";
        for (User r : recipients) {
            names = names + r.getName() + " ";
        }
        return "[" + sender.getName() + " -> " + names + "]: " + content;
    }
}

```

```java
package part2;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class ChatServer {
    private List<User> users;
    private Map<User, List<User>> blockedUsers;

    public ChatServer() {
        users = new ArrayList<>();
        blockedUsers = new HashMap<>();
    }

    public void registerUser(User user) {
        users.add(user);
        blockedUsers.put(user, new ArrayList<>());
    }

    public void unregisterUser(User user) {
        users.remove(user);
        blockedUsers.remove(user);
    }

    public void blockUser(User blocker, User blocked) {
        blockedUsers.get(blocker).add(blocked);
    }

    public boolean isBlocked(User receiver, User sender) {
        return blockedUsers.get(receiver).contains(sender);
    }

    public void sendMessage(Message message) {
        User sender = message.getSender();
        for (User recipient : message.getRecipients()) {
            if (!users.contains(recipient)) {
                System.out.println(recipient.getName() + " is not
registered. Message not delivered.");
            } else if (isBlocked(recipient, sender)) {
                System.out.println(recipient.getName() + " has blocked
 " + sender.getName() + ". Message not delivered.");
            } else {
                recipient.receiveMessage(message);
            }
        }
    }

    public void undoMessage(User sender, MessageMemento memento) {
        for (User user : users) {
            if (user != sender) {
                user.getChatHistory().removeMessage(memento);
```

```
52                }
53            }
54        }
55    }
```

File - /Users/mazenfraihat/IdeaProjects/HW10/src/main/java/part2/ChatHistory.java

```java
package part2;

import java.util.ArrayList;
import java.util.List;
import java.util.Stack;
import java.util.Iterator;

public class ChatHistory implements IterableByUser {
    private List<Message> messages;
    private Stack<MessageMemento> mementos;

    public ChatHistory() {
        messages = new ArrayList<>();
        mementos = new Stack<>();
    }

    public void addMessage(Message message) {
        messages.add(message);
    }

    public void saveMemento(MessageMemento memento) {
        mementos.push(memento);
    }

    public MessageMemento getLastMessageSent() {
        if (mementos.isEmpty()) {
            return null;
        }
        return mementos.pop();
    }

    public void removeMessage(MessageMemento memento) {
        for (int i = 0; i < messages.size(); i++) {
            Message m = messages.get(i);
            if (m.getContent().equals(memento.getContent())
                    && m.getTimestamp().equals(memento.getTimestamp
()())) {
                messages.remove(i);
                return;
            }
        }
    }

    public List<Message> getMessages() {
        return messages;
    }

    public Iterator<Message> iterator(User userToSearchWith) {
        return new searchMessagesByUser(messages, userToSearchWith);
    }
}


```

53
54
55

File - /Users/mazenfraihat/IdeaProjects/HW10/src/main/java/part2/IterableByUser.java

```java
package part2;

import java.util.Iterator;

public interface IterableByUser {
    Iterator<Message> iterator(User userToSearchWith);
}
```

File - /Users/mazenfraihat/IdeaProjects/HW10/src/main/java/part2/MessageMemento.java

```java
package part2;

import java.time.LocalDateTime;

public class MessageMemento {
    private String content;
    private LocalDateTime timestamp;

    public MessageMemento(String content, LocalDateTime timestamp) {
        this.content = content;
        this.timestamp = timestamp;
    }

    public String getContent() {
        return content;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }
}
```

```java
package part2;

import java.util.Iterator;
import java.util.List;

public class searchMessagesByUser implements Iterator<Message> {
    private List<Message> messages;
    private User user;
    private int index;

    public searchMessagesByUser(List<Message> messages, User user) {
        this.messages = messages;
        this.user = user;
        this.index = 0;
    }

    public boolean hasNext() {
        while (index < messages.size()) {
            Message m = messages.get(index);
            if (m.getSender() == user || m.getRecipients().contains(user)) {
                return true;
            }
            index++;
        }
        return false;
    }

    public Message next() {
        Message m = messages.get(index);
        index++;
        return m;
    }
}
```